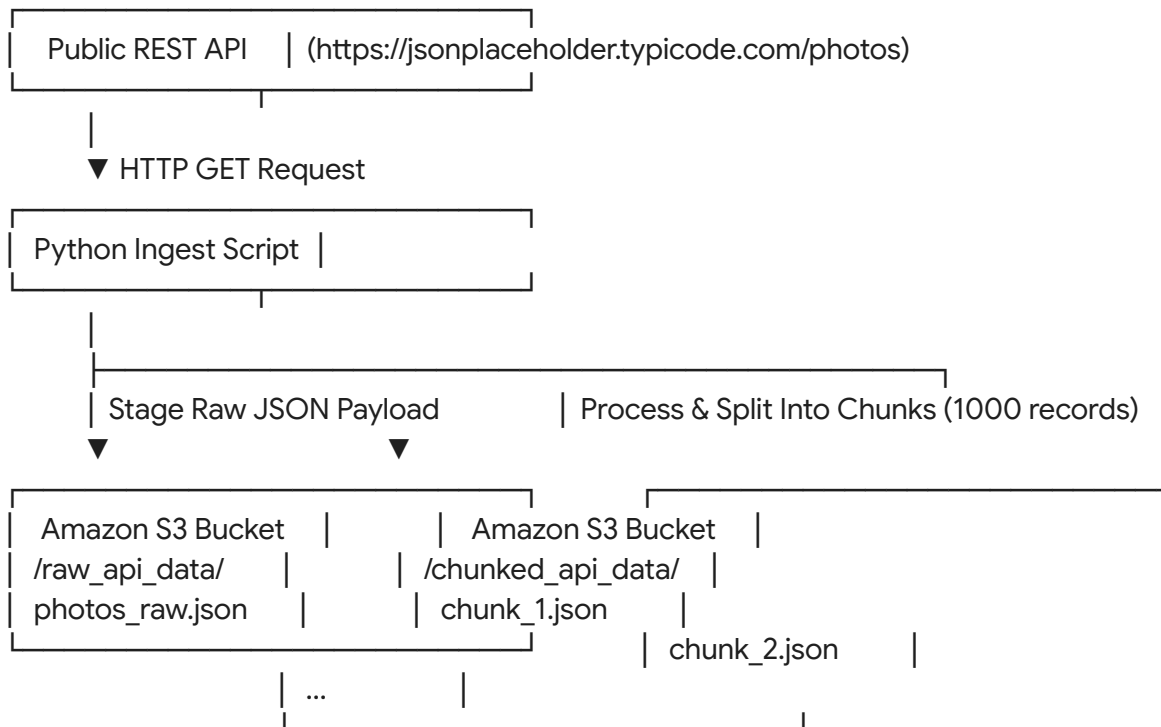


AWS S3 Pipeline: REST API Ingestion and JSON Chunking

1. Executive Summary

This document provides a step-by-step technical guide for building a data pipeline that fetches raw data from a public REST API, stages the full payload as a raw JSON file in Amazon S3, and subsequently processes and splits the data into smaller, manageable JSON chunk files (chunk_1.json, chunk_2.json, etc.) in another directory within the same S3 bucket.

2. Architecture & Data Flow



3. Prerequisites & Local Environment Setup

3.1 CLI Tools & Dependencies

- **AWS CLI (v2)** configured with valid credentials.
- **Python 3.8+**

3.2 Required Python Libraries

Install the required dependencies using pip:

```
pip install pandas requests s3fs
```

- **requests:** Handles HTTP communications with the external REST API.
- **pandas:** Used for DataFrame manipulation, slicing, and JSON formatting.
- **s3fs:** File-system interface that seamlessly handles streaming data between local Python execution environments and Amazon S3 URIs.

4. AWS S3 & IAM Configuration

4.1 S3 Bucket & Folder Setup

1. Open the **Amazon S3 Console**.
2. Locate or create your S3 bucket (e.g., big-file-chunks).
3. Create two folder prefixes inside the bucket:
 - raw_api_data/
 - chunked_api_data/

4.2 IAM Policy

Ensure your IAM User or Role has permissions to list, read, and write to the target bucket path.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "S3BucketListAccess",
      "Effect": "Allow",
      "Action": [
        "s3:ListBucket"
      ],
      "Resource": "arn:aws:s3:::big-file-chunks"
    },
    {
      "Sid": "S3ObjectReadWriteAccess",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3:::big-file-chunks/*"
    }
  ]
}
```

```
}
```

5. Python Implementation

Save the pipeline script as `api_to_s3_json_chunker.py`:

```
import json
import requests
import pandas as pd
import s3fs

# Pipeline Configuration
API_URL = "https://jsonplaceholder.typicode.com/photos" # 5,000 sample records
BUCKET_NAME = "big-file-chunks"
RAW_S3_PATH = f"s3://{BUCKET_NAME}/raw_api_data/photos_raw.json"
CHUNK_S3_DIR = f"s3://{BUCKET_NAME}/chunked_api_data"
CHUNK_SIZE = 1000

def fetch_and_stage_raw_api():
    """Fetch JSON data from REST API and save raw payload directly to S3."""
    print(f"1. Fetching data from API: {API_URL}")
    response = requests.get(API_URL)
    response.raise_for_status()
    data = response.json()
    print(f"   Successfully fetched {len(data)} records.")

    print(f"2. Uploading raw JSON to S3: {RAW_S3_PATH}")
    fs = s3fs.S3FileSystem()
    with fs.open(RAW_S3_PATH, "w") as f:
        json.dump(data, f, indent=2)
    print("   Raw JSON staging complete.")

def process_and_chunk_s3_json():
    """Read raw JSON from S3, split into chunks, and save as JSON files in output S3 folder."""
    print(f"3. Reading raw JSON from S3: {RAW_S3_PATH}")
    df = pd.read_json(RAW_S3_PATH)

    total_records = len(df)
    print(f"   Total records loaded into DataFrame: {total_records}")
```

```

print(f"4. Chunking data (Size: {CHUNK_SIZE}) and uploading to: {CHUNK_S3_DIR}")

for i in range(0, total_records, CHUNK_SIZE):
    chunk = df.iloc[i : i + CHUNK_SIZE]
    chunk_num = (i // CHUNK_SIZE) + 1
    target_file = f"{CHUNK_S3_DIR}/chunk_{chunk_num}.json"

    # Export DataFrame chunk directly to S3 as a formatted JSON file
    chunk.to_json(target_file, orient="records", indent=2)
    print(f"  Uploaded: {target_file} ({len(chunk)} records)")

print("Pipeline Execution Completed Successfully!")

if __name__ == "__main__":
    fetch_and_stage_raw_api()
    process_and_chunk_s3_json()

```

6. Code Detailed Walkthrough

1. **requests.get(API_URL):** Sends an HTTP GET request to pull the full dataset from the REST endpoint.
2. **s3fs.S3FileSystem():** Authenticates using the active environment credentials (AWS CLI configuration) and creates a direct file-like interface to S3.
3. **pd.read_json(RAW_S3_PATH):** Loads the staged raw JSON from S3 directly into a Pandas DataFrame.
4. **df.iloc[i : i + CHUNK_SIZE]:** Performs row-based slicing across the DataFrame without altering schema or column integrity.
5. **chunk.to_json(target_file, orient="records", indent=2):** Formats each slice as an array of JSON objects and writes the chunk file directly to `s3://big-file-chunks/chunked_api_data/chunk_N.json`.

7. Execution & Verification

Step 7.1: Run the Script

Execute the script from your terminal:

```
python api_to_s3_json_chunker.py
```

Step 7.2: Verify Outputs via AWS CLI

Verify the raw JSON staging file:

```
aws s3 ls s3://big-file-chunks/raw_api_data/
```

Verify the generated JSON chunk files:

```
aws s3 ls s3://big-file-chunks/chunked_api_data/
```

Expected terminal output:

```
2026-09-25 14:18:00 1024500 chunk_1.json
2026-09-25 14:18:01 1024800 chunk_2.json
2026-09-25 14:18:01 1025100 chunk_3.json
2026-09-25 14:18:02 1024900 chunk_4.json
2026-09-25 14:18:02 1024600 chunk_5.json
```

chunked_api_data/ [Copy S3 URI](#)

[Objects](#) | [Properties](#)

Objects (5)

[Refresh](#) [Copy S3 URI](#) [Copy URL](#) [Download](#) [Open](#) [Delete](#) [Actions](#) [Create folder](#) [Upload](#)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

< 1 >

⚙️

☐	Name	Type	Last modified	Size	Storage class
☐	chunk_1.json	json	September 25, 2026, 14:16:45 (UTC+05:30)	211.8 KB	Standard
☐	chunk_2.json	json	September 25, 2026, 14:16:45 (UTC+05:30)	212.6 KB	Standard
☐	chunk_3.json	json	September 25, 2026, 14:16:46 (UTC+05:30)	211.2 KB	Standard
☐	chunk_4.json	json	September 25, 2026, 14:16:47 (UTC+05:30)	212.1 KB	Standard
☐	chunk_5.json	json	September 25, 2026, 14:16:47 (UTC+05:30)	213.3 KB	Standard

Objects (1)

[Refresh](#) [Copy S3 URI](#) [Copy URL](#) [Download](#) [Open](#) [Delete](#) [Actions](#) [Create folder](#) [Upload](#)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 Inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

< 1 >

Name	Type	Last modified	Size	Storage class
photos_raw.json	json	September 25, 2026, 14:16:41 (UTC+05:30)	1.0 MB	Standard